Inheritance in Java

Definition

Inheritance is an object-oriented programming mechanism by which one class acquires the accessible properties and methods of another class.

Inheritance enables **code reusability**, **method overriding**, and **runtime polymorphism**.

The class whose properties are inherited is called the **superclass**, while the class that inherits them is called the **subclass**.

Important terminology

Term	Meaning
Superclass	The class whose properties and methods are inherited
Subclass	The class that inherits from another class
Base class	Another name for superclass
Derived class	Another name for subclass
`extends`	Keyword used to establish class inheritance
IS-A relationship	Relationship represented by inheritance

Syntax

```
class SuperClass {
    // Fields and methods
}

class SubClass extends SuperClass {
    // Additional fields and methods
}
```

Example

```
class Animal {
    void eat() {
```

```
        System.out.println("Animal eats food");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();

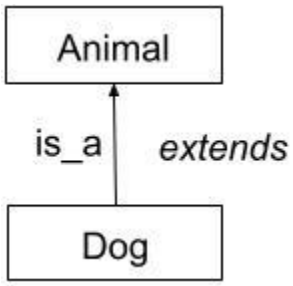
        d.eat();
        d.bark();
    }
}
```

Output:

```
Animal eats food
Dog barks
```

Explanation

1. Animal is the superclass.
2. Dog is the subclass.
3. extends establishes the inheritance relationship.
4. The Dog class inherits the accessible eat() method from Animal.
5. An object of Dog can invoke both eat() and bark().



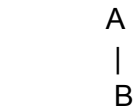
2. Types of Inheritance

Java supports the following inheritance structures.

2.1 Single inheritance

In single inheritance, one subclass inherits from one superclass.

Diagram:



Example:

```
class A {
    void displayA() {
        System.out.println("Class A");
    }
}

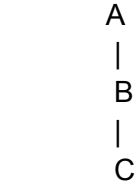
class B extends A {
    void displayB() {
        System.out.println("Class B");
    }
}
```

Here, class B inherits from class A.

2.2 Multilevel inheritance

In multilevel inheritance, a class inherits from another class that is itself a subclass.

Diagram:



Example:

```
class Animal {
    void eat() {
        System.out.println("Eating");
```

```
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Barking");
    }
}

class Puppy extends Dog {
    void weep() {
        System.out.println("Weeping");
    }
}

public class Main {
    public static void main(String[] args) {
        Puppy p = new Puppy();

        p.eat();
        p.bark();
        p.weep();
    }
}
```

Output:

```
Eating
Barking
Weeping
```

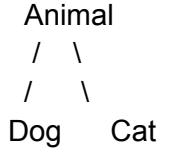
Explanation

The class Puppy inherits the accessible methods of both Dog and Animal.

2.3 Hierarchical inheritance

In hierarchical inheritance, multiple subclasses inherit from a common superclass.

Diagram:



Example:

```
class Animal {
    void eat() {
        System.out.println("Eating");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Barking");
    }
}

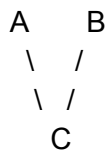
class Cat extends Animal {
    void meow() {
        System.out.println("Meowing");
    }
}
```

Both Dog and Cat inherit the accessible eat() method.

2.4 Multiple inheritance

Multiple inheritance occurs when one class inherits from more than one superclass.

Diagram:



Java does not support multiple inheritance of classes.

The following code is invalid:

```
class A {
}

class B {
}

class C extends A, B {
}
```

Reason:

Multiple inheritance of classes can introduce ambiguity when two parent classes contain methods with the same signature.

This is commonly illustrated using the diamond problem.

Java avoids such class-inheritance ambiguity by allowing a class to extend only one superclass.

However, a Java class can implement multiple interfaces.

```
interface A {
    void show();
}

interface B {
    void display();
}

class C implements A, B {
    public void show() {
        System.out.println("A");
    }

    public void display() {
        System.out.println("B");
    }
}
```

2.5 Hybrid inheritance

Hybrid inheritance is a combination of two or more inheritance structures.

Java does not support hybrid inheritance involving multiple superclass inheritance.

However, hybrid inheritance structures can be implemented through a combination of class inheritance and interfaces.

Comparison of inheritance types

Type	Description	Supported in Java
Single	One subclass inherits one superclass	Yes
Multilevel	A chain of inherited classes	Yes

Hierarchical	Multiple subclasses inherit one superclass	Yes
Multiple	One class inherits multiple classes	No
Hybrid	Combination of inheritance structures	Through interfaces

Review questions

2 Marks

1. List the different types of inheritance.
2. Why does Java not support multiple inheritance of classes?

4 Marks

1. Explain the types of inheritance supported in Java with diagrams.
2. Explain how multiple inheritance can be achieved using interfaces.

---

3. The super Keyword

Definition

The super keyword in Java is used to refer to members of the immediate superclass from within a subclass.

Uses of super

The super keyword has three important uses:

1. Access superclass variables.
2. Invoke superclass methods.
3. Invoke superclass constructors.

3.1 Accessing superclass variables

When the superclass and subclass contain variables with the same name, super can be used to access the superclass variable.

Example

```
class Animal {
    String color = "White";
}

class Dog extends Animal {
    String color = "Black";

    void display() {
        System.out.println(color);
        System.out.println(super.color);
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.display();
    }
}
```

Output:

```
Black
White
```

Explanation

- color refers to the variable in Dog.
- super.color refers to the variable in Animal.

3.2 Calling superclass methods

If a subclass overrides a superclass method, the superclass version can be invoked using super.

Example

```
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}
```

```
class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }

    void display() {
        sound();
        super.sound();
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.display();
    }
}
```

Output:

```
Dog barks
Animal makes sound
```

Explanation

- sound() invokes the overridden method of Dog.
- super.sound() invokes the superclass implementation.

3.3 Calling superclass constructors

The statement super() is used to invoke a constructor of the immediate superclass.

Example

```
class Animal {
    Animal() {
        System.out.println("Animal constructor");
    }
}

class Dog extends Animal {
    Dog() {
        super();
        System.out.println("Dog constructor");
    }
```

```
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
    }
}
```

Output:

```
Animal constructor
Dog constructor
```

Rules for super()

1. super() invokes a superclass constructor.
2. If explicitly written, it must be the first statement of the constructor body.
3. If a constructor does not explicitly invoke super() or this(), Java normally inserts super() automatically.
4. super() can invoke a parameterized superclass constructor by passing arguments.
5. The superclass constructor executes before the subclass constructor body.

Important university question

Question: What is the use of the super keyword? Give an example. (2 Marks)

Answer:

The super keyword refers to members of the immediate superclass. It is used to access superclass variables, invoke superclass methods, and call superclass constructors.

Example:

```
class A {
    void show() {
        System.out.println("Class A");
    }
}

class B extends A {
    void show() {
        super.show();
        System.out.println("Class B");
    }
}
```

This is Question 12(b) from the November 2025 university examination. PBCST304 OBJECT ORIENTED PROGRAMMING

5. Calling Order of Constructors in Inheritance

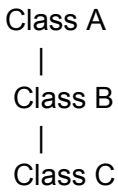
Definition

When an object of a subclass is created, constructors execute from the superclass to the subclass.

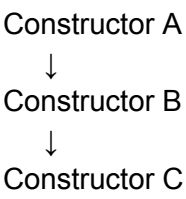
This is called constructor chaining.

Constructor execution order

For multilevel inheritance:



When an object of C is created:



Example

```
class A {
    A() {
        System.out.println("Constructor A");
    }
}

class B extends A {
    B() {
        System.out.println("Constructor B");
    }
}
```

```
class C extends B {
    C() {
        System.out.println("Constructor C");
    }
}

public class Main {
    public static void main(String[] args) {
        C obj = new C();
    }
}
```

Output:

```
Constructor A
Constructor B
Constructor C
```

Explanation

When new C() executes:

1. The constructor of C is invoked.
2. It invokes the superclass constructor of B.
3. The constructor of B invokes the superclass constructor of A.
4. Constructor A completes first.
5. Constructor B completes next.
6. Constructor C completes last.

Important: The superclass constructor is executed before the subclass constructor body.

Consider a superclass Parent containing an instance variable name. The parameterized constructor initializes this variable.

The subclass Child uses super("Arun") to invoke the superclass constructor and initialize the inherited variable.

```
class Parent {
    String name;  // Instance variable

    // Parameterized constructor
    Parent(String name) {
        this.name = name;
        System.out.println("Parent constructor called");
    }
```

```
    void displayName() {
        System.out.println("Name: " + name);
    }
}

class Child extends Parent {

    Child() {
        super("Arun");  // Calls Parent(String name)
        System.out.println("Child constructor called");
    }
}

public class Main {
    public static void main(String[] args) {
        Child c = new Child();
        c.displayName();
    }
}
Output
Parent constructor called
Child constructor called
Name: Arun
```

Explanation

Step 1: Creating the subclass object

Child c = new Child();

When a Child object is created, its constructor is invoked.

Step 2: Calling the superclass constructor

Child() {

  super("Arun");

  System.out.println("Child constructor called");

}

The statement super("Arun") invokes the parameterized constructor of Parent.

Step 3: Initializing the superclass instance variable

```
Parent(String name) {

    this.name = name;

    System.out.println("Parent constructor called");

}
```

Here:

- The parameter name receives "Arun".
- this.name refers to the instance variable of the superclass.
- The statement this.name = name initializes the instance variable with "Arun".

Step 4: Executing the subclass constructor

After the superclass constructor completes, control returns to the Child constructor.

```
System.out.println("Child constructor called");
```

Step 5: Accessing the inherited method

```
c.displayName();
```

The Child object invokes the inherited method displayName(), which displays the initialized instance variable.

Why Is super("Arun") Required?

The superclass Parent contains only a parameterized constructor:

```
Parent(String name) {

    this.name = name;

}
```

It does not contain a no-argument constructor.

If we write:

```
class Child extends Parent {

    Child() {
```

```
        System.out.println("Child constructor called");

    }

}
```

Compilation error occurs.

Reason: Java implicitly attempts to invoke super(), but the superclass does not have a no-argument constructor.

Therefore, we must explicitly invoke the appropriate parameterized constructor:

super("Arun");

Important Points for University Examination

1. Constructors are not inherited by subclasses.
2. The super() statement invokes a superclass constructor.
3. A parameterized superclass constructor can be invoked using super(arguments).
4. The superclass constructor executes before the subclass constructor body.
5. If the superclass does not have a no-argument constructor, the subclass must explicitly invoke an accessible superclass constructor with appropriate arguments.

Key concept: Constructor chaining ensures that the superclass portion of an object is initialized before the subclass constructor body executes.

Common examination question

Predict the output:

```
class A {
    A() {
        System.out.println("A");
    }
}

class B extends A {
    B() {
        System.out.println("B");
    }
}
```

```
class C extends B {
    C() {
        System.out.println("C");
    }
}

public class Test {
    public static void main(String[] args) {
        new C();
    }
}
```

Answer:

```
A
B
C
```

Reason: Constructors execute from the highest superclass to the most derived subclass.

---

6. Method Overriding

Definition

Method overriding occurs when a subclass provides its own implementation of an instance method already defined in its superclass.

The overriding method must have a compatible signature and return type.

Purpose

Method overriding allows a subclass to provide specialized behavior for an inherited method.

Example

```
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}

class Dog extends Animal {
```

```
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.sound();
    }
}
```

Output:

```
Dog barks
```

Explanation

1. Animal defines the method sound().
2. Dog inherits from Animal.
3. Dog overrides sound().
4. When sound() is invoked on a Dog object, the subclass implementation executes.

Rules of method overriding

1. Overriding requires an inheritance relationship.
2. The overriding method must have the same name and parameter list.
3. The return type must be the same or covariant.
4. The access level of the overriding method cannot be more restrictive.
5. Private methods cannot be overridden.
6. Final methods cannot be overridden.
7. Static methods are hidden, not overridden.
8. Constructors cannot be overridden.
9. The @Override annotation helps the compiler verify correct overriding.

Method overloading vs method overriding

Method Overloading	Method Overriding
Same method name with different parameters	Same method name and parameters
Inheritance is not required	Inheritance is required
Compile-time polymorphism	Runtime polymorphism

Methods have different parameter lists	Parameters must match
May occur within one class	Occurs between superclass and subclass
Resolved using argument types at compile time	Instance implementation selected at runtime

University examination question

Write a Java program demonstrating method overriding. (4 Marks)

The example above is suitable for this question.

For a 4-mark answer, include:

- Definition of method overriding.
- Java program.
- Output.
- Explanation of how the subclass method replaces the inherited behavior.

---

7. Dynamic Method Dispatch

Definition

Dynamic Method Dispatch is the mechanism by which a call to an overridden instance method is resolved at runtime based on the actual object type.

It is the mechanism used to implement runtime polymorphism in Java.

Important concept

A superclass reference can refer to a subclass object.

```
Animal a = new Dog();
```

Here:

- Animal is the reference type.
- Dog is the actual object type.
- The overriding method that executes is selected according to the actual object type.

Example

```
class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

class Cat extends Animal {
    @Override
    void sound() {
        System.out.println("Cat meows");
    }
}

public class Main {
    public static void main(String[] args) {

        Animal a;

        a = new Animal();
        a.sound();

        a = new Dog();
        a.sound();

        a = new Cat();
        a.sound();
    }
}
```

Output:

```
Animal sound
Dog barks
Cat meows
```

Explanation

Initially:

```
a = new Animal();
a.sound();
```

The reference points to an Animal object.

Therefore, Animal.sound() executes.

Next:

```
a = new Dog();
a.sound();
```

The same reference now points to a Dog object.

Therefore, Dog.sound() executes.

Finally:

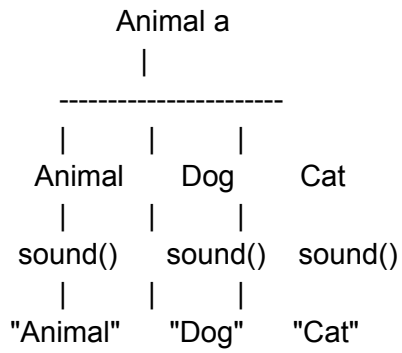
```
a = new Cat();
a.sound();
```

The reference points to a Cat object.

Therefore, Cat.sound() executes.

Diagram

Superclass reference: Animal a



Important points

- Dynamic method dispatch is based on method overriding.
- Method selection occurs at runtime.
- It requires a superclass reference pointing to an object whose class provides an overriding implementation.

- Only eligible instance methods participate in dynamic method dispatch.
- Static methods are not dynamically dispatched in this way.
- The reference type determines which methods are accessible at compile time.
- The actual object type determines which overriding implementation executes at runtime.

Example demonstrating reference-type restrictions

```
class Animal {
    void eat() {
        System.out.println("Eating");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Barking");
    }
}

public class Main {
    public static void main(String[] args) {

        Animal a = new Dog();

        a.eat();

        // a.bark(); // Compilation error
    }
}
```

Even though a refers to a Dog object, it cannot directly invoke bark() because bark() is not declared in Animal.

Review questions

2 Marks

1. Define dynamic method dispatch.
2. What is runtime polymorphism?

4 Marks

1. Explain dynamic method dispatch with a Java program.
2. Explain how method overriding supports runtime polymorphism.

---

8. Using final with Inheritance

Definition

The keyword final is used to restrict modification, overriding, or inheritance depending on where it is applied.

The final keyword can be used with:

1. Variables
2. Methods
3. Classes

8.1 Final variables

A final variable can be assigned only once.

Example

```
class Test {
    final int MAX = 100;

    void display() {
        System.out.println(MAX);
    }
}
```

The following reassignment is invalid:

```
MAX = 200;
```

A final variable cannot be reassigned after initialization.

Note: If a final variable contains an object reference, the referenced object's state may still be modified, provided the object is mutable.

8.2 Final methods

A final method cannot be overridden by a subclass.

Example

```
class Parent {
    final void display() {
```

```
        System.out.println("Parent method");
    }
}

class Child extends Parent {
    // Cannot override display()
}
```

The following declaration would cause a compilation error:

```
class Child extends Parent {
    void display() {
        System.out.println("Child method");
    }
}
```

Why are final methods used?

Final methods prevent subclasses from replacing an existing method implementation.

They are useful when certain behavior must remain unchanged.

8.3 Final classes

A final class cannot be extended.

Example

```
final class A {
    void display() {
        System.out.println("Class A");
    }
}
```

The following code is invalid:

```
class B extends A {
}
```

Explanation

Since A is declared final, no subclass can inherit from it.

Example from Java

The String class is final.

public final class String

Therefore, a class cannot extend String.

Comparison

Declaration	Restriction
final int x = 10;	Variable cannot be reassigned
final void show()	Method cannot be overridden
final class A	Class cannot be extended

University examination question

Explain the use of the final keyword in Java. (2 Marks)

Model answer:

The final keyword restricts modification or extension. A final variable cannot be reassigned, a final method cannot be overridden, and a final class cannot be inherited.

Example:

```
final class A {
}
```

The class A cannot be extended by another class.

This question appeared as Question 4 in the November 2025 university examination.
PBCST304 OBJECT ORIENTED PROGRAMMING

---

9. Important University Previous-Year Questions

The following questions are from the uploaded PBCST304 November 2025 examination.

Question 1: Predict the output and justify (2 Marks)

November 2025 – Part A, Question 3

```
class Animal {
    Animal() {
        System.out.println("Animal constructor called");
    }

    void sound() {
        System.out.println("Animals make sounds");
    }
}

class Dog extends Animal {
    Dog() {
        super();
        System.out.println("Dog constructor called");
    }

    void sound() {
        System.out.println("Dog barks");
    }
}

public class InheritanceDemo {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.sound();
    }
}
```

Answer

```
Animal constructor called
Dog constructor called
Dog barks
```

Justification

1. Creating a Dog object invokes the Dog constructor.
2. super() invokes the Animal constructor first.
3. After the superclass constructor completes, the Dog constructor body executes.
4. Calling d.sound() invokes the overridden method in Dog.

Concepts tested: Constructor chaining, super(), and method overriding. PBCST304 OBJECT ORIENTED PROGRAMMING

Question 2: Inheritance using constructors (6 Marks)

November 2025 – Part B, Question 11

Question: Write a Java program to demonstrate inheritance using constructors.

The problem specifies a Bank superclass and two subclasses, SBAccount and CurrentAccount.

The superclass contains:

- bankName
- branch
- A parameterized constructor
- displayBankDetails()

Both subclasses contain an account holder and balance. Their constructors must initialize inherited attributes using super(), and their display methods must print complete account details. PBCST304 OBJECT ORIENTED PROGRA…

Model answer

```
class Bank {
    String bankName;
    String branch;

    Bank(String bankName, String branch) {
        this.bankName = bankName;
        this.branch = branch;
    }

    void displayBankDetails() {
        System.out.println("Bank: " + bankName);
        System.out.println("Branch: " + branch);
    }
}

class SBAccount extends Bank {
    String accountHolder;
    double balance;

    SBAccount(String bankName, String branch,
```

```
                String holder, double balance) {

        super(bankName, branch);

        this.accountHolder = holder;
        this.balance = balance;
    }

    void displaySBDetails() {
        displayBankDetails();

        System.out.println("Account Holder: "
                           + accountHolder);

        System.out.println("Balance: " + balance);
    }
}

class CurrentAccount extends Bank {
    String accountHolder;
    double balance;

    CurrentAccount(String bankName, String branch,
                   String holder, double balance) {

        super(bankName, branch);

        this.accountHolder = holder;
        this.balance = balance;
    }

    void displayCurrentDetails() {
        displayBankDetails();

        System.out.println("Account Holder: "
                           + accountHolder);

        System.out.println("Balance: " + balance);
    }
}

public class BankDemo {
    public static void main(String[] args) {

        SBAccount sb = new SBAccount(
```

```
            "SBI", "Kasaragod", "Arun", 50000);

        CurrentAccount ca = new CurrentAccount(
            "SBI", "Kasaragod", "Anu", 80000);

        System.out.println("Savings Account");
        sb.displaySBDetails();

        System.out.println();

        System.out.println("Current Account");
        ca.displayCurrentDetails();
    }
}
```

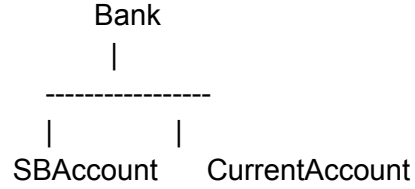
Output

```
Savings Account
Bank: SBI
Branch: Kasaragod
Account Holder: Arun
Balance: 50000.0

Current Account
Bank: SBI
Branch: Kasaragod
Account Holder: Anu
Balance: 80000.0
```

Explanation

The program demonstrates hierarchical inheritance.



Both subclasses inherit from Bank.

The super() statement invokes the parameterized constructor of Bank, initializing the superclass attributes.

The subclass constructors then initialize their own attributes.

Question 3: Runtime polymorphism (4 Marks)

November 2025 – Part B, Question 12(a)

Question: Write a Java program to demonstrate runtime polymorphism using method overriding. PBCST304 OBJECT ORIENTED PROGRA…

Model answer

```
class Shape {
    void draw() {
        System.out.println("Drawing Shape");
    }
}

class Circle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing Circle");
    }
}

class Rectangle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing Rectangle");
    }
}

public class Main {
    public static void main(String[] args) {

        Shape s;

        s = new Circle();
        s.draw();

        s = new Rectangle();
        s.draw();
    }
}
```

Output

```
Drawing Circle
Drawing Rectangle
```

Explanation

The superclass reference s refers to different subclass objects.

The draw() method is selected at runtime based on the actual object.

This demonstrates runtime polymorphism using method overriding and dynamic method dispatch.
